

Referência Técnica — AutoTyper

Visão Geral

AutoTyper simula pressionamentos de tecla em qualquer janela ativa do SO, digitando texto caractere por caractere com intervalo configurável. Suporta marcadores embutidos no texto (`[[pause:N]]`, `[[speed:N]]`, `[[key:F5]]`) e modo por linha para scripts interativos. Um assistente de IA opcional (Anthropic Claude) gera o texto a digitar a partir de um prompt em linguagem natural.

Estrutura do Pacote

```
typer.py          - shim de compatibilidade: importa e executa
autotyper.__main__.main()
autotyper/
├─ __init__.py    - docstring do pacote, sem exports
├─ __main__.py    - entry point: parse_args() → run_headless() ou
TyperApp.mainloop()
├─ config.py      - TRANSLATIONS, SPEED_PROFILES, StatusStyle,
platform_mono_font(), ModelInfo, AI_FALLBACK_MODELS, ai_system_prompt()
├─ markers.py     - Instruction, parse_instructions(), _MARKER_RE,
_SPECIAL_KEYS
├─ engine.py      - TypingEngine, TypingCallbacks (sem dependência de
tkinter)
├─ ai.py          - AIGenerator, AICallbacks, build_client(),
has_credentials() (sem dependência de tkinter)
├─ app.py         - TyperApp (GUI, tkinter/ttkbootstrap)
└─ cli.py         - parse_args(), run_headless()
```

Módulos

autotyper/config.py

Símbolo	Tipo	Descrição
TRANSLATIONS	dict[str, dict[str, str]]	Todas as strings visíveis ao usuário em EN e PT
SPEED_PROFILES	list[tuple[int None, str]]	Pares (ms, chave_tradução) para o combo de perfil
StatusStyle	enum.StrEnum	Constantes de estilo ttkbootstrap: IDLE, SUCCESS, WARNING, ERROR
platform_mono_font(size)	func → tuple[str, int]	Fonte monoespaçada por plataforma (Consolas / Menlo / Monospace)

Símbolo	Tipo	Descrição
<code>AI_MODEL</code>	<code>str</code>	Id de modelo usado como último recurso ("claude-opus-4-8")
<code>ModelInfo</code>	<code>dataclass</code>	<code>id</code> , <code>display_name</code> , <code>supports_adaptive: bool</code> , <code>effort_levels: tuple[str, ...]</code>
<code>AI_FALLBACK_MODELS</code>	<code>list[ModelInfo]</code>	Lista estática usada só quando a Models API não pode ser consultada (sem <code>anthropic</code> , sem credencial, ou erro de rede)
<code>ai_system_prompt(lang)</code>	<code>func → str</code>	System prompt do assistente, com nota de idioma para os comentários

autotyper/ai.py

Sem dependência de tkinter — espelha `engine.py`. Roda a chamada à API numa daemon thread e reporta via `AICallbacks`.

Símbolo	Tipo	Descrição
<code>is_available()</code>	<code>func → bool</code>	True se o pacote <code>anthropic</code> está instalado
<code>has_credentials()</code>	<code>func → bool</code>	True se há credencial (env, perfil <code>ant</code> , ou login do Claude Code)
<code>is_auth_error(e)</code>	<code>func → bool</code>	True se a exceção é <code>anthropic.AuthenticationError</code>
<code>build_client()</code>	<code>func → anthropic.Anthropic</code>	Resolve credenciais (ver <i>Resolução de Credenciais</i>)
<code>list_models()</code>	<code>func → list[ModelInfo]</code>	Busca todos os modelos visíveis via <code>client.models.list()</code> , sem filtragem; lê <code>supports_adaptive/effort_levels</code> de <code>capabilities</code>
<code>AICallbacks</code>	<code>dataclass</code>	<code>on_start</code> , <code>on_text(chunk)</code> , <code>on_done(full)</code> , <code>on_error(e)</code> — chamados da thread worker
<code>AIGenerator</code>	<code>class</code>	<code>generate(prompt, lang, model, callbacks, supports_adaptive, effort)</code> inicia o streaming em daemon thread; <code>is_generating: bool</code>

Resolução de Credenciais (`build_client()`)

Ordem de precedência, **sem armazenar nada localmente**:

1. Env `ANTHROPIC_API_KEY` / `ANTHROPIC_AUTH_TOKEN` → SDK resolve (`anthropic.Anthropic()`)
2. Perfil `ant auth login` em `~/.config/anthropic/` → SDK resolve

3. Login do Claude Code / VS Code em `~/.claude/.credentials.json` →
`claudeAiOAuth.accessToken` usado como token OAuth Bearer com header `anthropic-beta: oauth-2025-04-20`

O token do item 3 é lido **a cada geração** (o Claude Code o renova no arquivo, então re-ler pega o token atualizado). Usa a cota da assinatura Claude, compartilhada com o Claude Code — 429 sob uso concorrente pesado.

Thinking condicional e nível de effort

`_run()` só inclui `thinking={"type": "adaptive"}` quando `supports_adaptive=True` é passado pelo chamador, e só inclui `output_config={"effort": ...}` quando um `effort` é passado. Esses dois valores vêm do `ModelInfo` do modelo selecionado (capacidades lidas dinamicamente via `list_models()`, não mais de um set fixo no código) — assim um modelo que não aceita adaptive thinking ou o parâmetro `effort` (ex.: Haiku 4.5, que rejeita ambos com HTTP 400) simplesmente não os recebe.

Lista de modelos dinâmica

`TyperApp` inicia com `AI_FALLBACK_MODELS` no combobox e, no fim do `__init__`, dispara `_refresh_ai_models_async()` numa thread daemon chamando `ai.list_models()`. Se a chamada tiver sucesso, `_apply_fetched_models()` substitui a lista (via `self.after(0, ...)`) sem filtrar nada — qualquer modelo novo ou mudança de versão aparece no próximo start sem alterar código. Falha de rede/credencial/lib mantém a lista estática em silêncio. O combobox de effort (`_ai_effort_combo`) é recalculado a cada troca de modelo (`_update_effort_options()`), mostrando só os níveis que aquele modelo suporta; fica desabilitado quando o modelo não suporta `effort`.

autotyper/markers.py

Símbolo	Tipo	Descrição
<code>Instruction</code>	<code>TypeAlias</code>	<code>tuple[str, str float None]</code> — um item da lista de instruções
<code>parse_instructions(text)</code>	<code>func → list[Instruction]</code>	Converte texto bruto em lista de instruções tipadas
<code>_MARKER_RE</code>	<code>re.Pattern</code>	Regex que reconhece <code>[[diretiva:valor]]</code>
<code>_SPECIAL_KEYS</code>	<code>dict[str, Key]</code>	Mapa de nome de tecla → constante <code>pynput.Key</code> . Inclui aliases comuns (<code>del</code> →delete, <code>pgup/pgdn</code> →page up/down, <code>return</code> →enter, <code>escape</code> →esc, <code>ins</code> →insert) — sem eles, um acorde como <code>ctrl+alt+del</code> teria a tecla <code>del</code> silenciosamente descartada

Formato de instrução

Tupla	Significado
<code>('char', str)</code>	Digitar este caractere
<code>('pause', float)</code>	Aguardar N segundos (só aceito se $N > 0$)
<code>('speed', float)</code>	Alterar intervalo para N ms (só aceito se $N > 0$)
<code>('speed', None)</code>	Restaurar intervalo base
<code>('key', str)</code>	Pressionar tecla ou combinação (<code>'ctrl+c'</code> , <code>'F5'</code> , <code>'win'</code>)

Marcadores com valor numérico ≤ 0 , formato inválido ou diretiva desconhecida são silenciosamente descartados.

autotyper/engine.py

TypingCallbacks

Dataclass com todos os callbacks opcionais. Todos são chamados da **thread worker** — o chamador é responsável por despachar para a thread principal se necessário.

Campo	Assinatura	Quando
<code>on_session_start</code>	<code>(dt_str: str, total: int)</code>	Antes do wait inicial
<code>on_waiting</code>	<code>(wait_s: float)</code>	Ao iniciar o período de espera
<code>on_progress</code>	<code>(done: int, total: int)</code>	A cada lote de chars digitados (modo normal)
<code>on_chunk_done</code>	<code>(cur: int, total: int)</code>	Após cada linha (modo chunk)
<code>on_done</code>	<code>(stopped: bool, done: int, total: int)</code>	Ao encerrar a sessão
<code>on_error</code>	<code>(e: Exception)</code>	Em caso de exceção no worker
<code>on_stop_requested</code>	<code>()</code>	Quando ESC é pressionado dentro do engine

TypingEngine

Membro	Descrição
<code>start(text, wait_s, interval_s, chunk_mode, callbacks)</code>	Inicia o worker em daemon thread. <code>interval_s</code> é em segundos
<code>stop()</code>	Sinaliza parada imediata (idempotente)
<code>toggle_pause() → bool</code>	Alterna pause/resume; retorna <code>True</code> se agora pausado

Membro	Descrição
<code>is_typing: bool</code>	Propriedade — True enquanto o worker estiver ativo
<code>is_paused: bool</code>	Propriedade — True enquanto pausado

Nota de contrato: `interval_s` está em segundos. Converta `ms` → `s` antes de chamar (`interval_ms / 1000.0`).

autotyper/app.py — TyperApp

Método	Thread	Descrição
<code>__init__(lang, initial_file, interval_ms, wait_s)</code>	Principal	Constrói a janela; <code>interval_ms</code> é em ms (convertido internamente)
<code>start_typing_thread()</code>	Principal	Lê widgets, constrói <code>TypingCallbacks</code> , chama <code>engine.start()</code>
<code>generate_with_ai()</code>	Principal	Lê prompt, modelo e effort; valida credenciais, constrói <code>AICallbacks</code> e chama <code>AIGenerator.generate()</code>
<code>_ai_on_start/_text/_done/_error</code>	Principal (via after)	Handlers de streaming: limpa editor no 1º chunk, insere texto, reabilita controles, mapeia erro de auth
<code>_selected_model_info()</code>	Principal	Retorna o <code>ModelInfo</code> escolhido no combobox (fallback: primeiro da lista)
<code>_selected_model()</code>	Principal	Retorna o <code>id</code> do modelo escolhido
<code>_selected_effort()</code>	Principal	Retorna o nível de effort escolhido, ou <code>None</code> se o modelo não suporta
<code>_refresh_ai_models_async()</code>	Principal (dispara thread)	Busca <code>ai.list_models()</code> em background e aplica o resultado via <code>after(0, ...)</code>
<code>request_stop()</code>	Qualquer	Para o engine; seguro chamar de qualquer thread
<code>_toggle_pause()</code>	Principal	Delega ao engine e atualiza botão
<code>_on_chunk_done(cur, total)</code>	Principal (via after)	Atualiza log, status e barra de progresso após cada linha
<code>_on_typing_error(e)</code>	Principal (via after)	Exibe erro no status e no log
<code>_on_typing_done(stopped, done, total)</code>	Principal (via after)	Atualiza log, status e reseta UI

Método	Thread	Descrição
<code>update_status(text, style)</code>	Qualquer	Thread-safe via <code>after(0, ...)</code>
<code>t(key, **kwargs)</code>	Qualquer	Lookup de tradução; retorna <code>[chave]</code> se ausente

Regra de thread safety: nenhum widget é lido ou escrito de fora da thread principal. Os callbacks recebem dados primitivos e delegam modificações de UI via `self.after(0, ...)`.

CollapsibleFrame (widget auxiliar em `app.py`)

Seção recolhível (`ttk.Frame`) usada pelos painéis **Assistente IA**, **Bitwarden** e **Log de Sessão**, todos iniciando **recolhidos** para manter a janela compacta. Um botão-cabeçalho (estilo `-link`) alterna a visibilidade do corpo `self.body` — onde os widgets-filhos são criados. `set_title(text)` atualiza o texto do cabeçalho; por isso `_refresh_ui_text()` chama `self._af.set_title(...)` / `self._bf.set_title(...)` / `self._log_frame.set_title(...)` em vez de `.configure(text=...)`.

Ordem de empacotamento (layout)

O rodapé — barra de status, log, barra de progresso e contador de caracteres — é empacotado com `side="bottom"` **antes** da área de texto (o único widget com `expand=True`), lá no `__init__`. Consequência: ao reduzir a janela, só a área de texto encolhe; o rodapé mantém altura fixa e a barra de status (empacotada primeiro, portanto a última a ceder espaço) nunca é cortada.

autotyper/cli.py

Símbolo	Descrição
<code>parse_args()</code> → Namespace	Argparse com flags <code>--file</code> , <code>--interval</code> , <code>--wait</code> , <code>--lang</code> , <code>--headless</code>
<code>run_headless(file_path, interval_ms, wait_s)</code>	Modo sem GUI; <code>interval_ms</code> é em ms (dividido por 1000 antes de passar ao engine)

Modelo de Threading

```

Thread principal (Tkinter event loop)
├── engine.start(...)
│   └── Thread worker (daemon)
│       ├── _run()
│       │   ├── _type_all()      – normal mode
│       │   └── _type_by_chunks() – chunk mode
│       └── callbacks → self.after(0, fn) → thread principal
├── _stop_event.set() / _pause_event.clear()  ← UI → worker
└── self.after(0, callback)                  ← worker → UI

```

Evento	Estado inicial	Semântica
<code>_stop_event</code>	cleared	<code>set()</code> = parar imediatamente
<code>_pause_event</code>	set	<code>set()</code> = rodando; <code>clear()</code> = pausado

`_interruptible_sleep(seconds)` dorme em chunks de $\leq 20\text{ms}$, testando `_stop_event` e `_pause_event` a cada iteração. Latência máxima de reação ao stop/pause: $\sim 50\text{ms}$.

Marcadores — Fluxo de Parsing

```

texto bruto
  ↓ parse_instructions(text)
list[Instruction]
  ↓ engine._type_all() / _type_by_chunks()
ações de teclado + sleeps

```

`[[speed:N]]` armazena N em ms na instrução. O engine converte para segundos no momento de aplicar (`val / 1000.0`), mantendo `base_interval` (em s) como referência para `[[speed:reset]]`.

Perfis de Velocidade

```

SPEED_PROFILES = [
    (200, "prof_slow"),    # Lento   / Slow
    (100, "prof_normal"),  # Normal
    (50,  "prof_fast"),    # Rápido / Fast
    (10,  "prof_turbo"),   # Turbo
    (None, "prof_custom"), # Personalizado / Custom
]

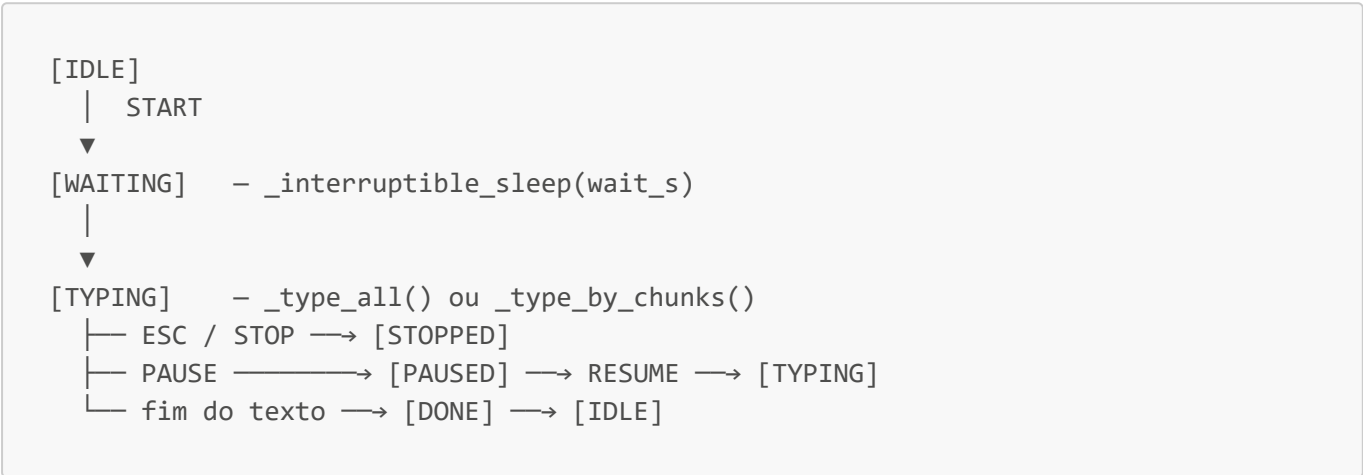
```

Selecionar um perfil atualiza o campo de intervalo. Editar o campo manualmente seleciona "Custom" se o valor não corresponder a nenhum perfil.

Modo Linha por Linha (Chunk Mode)

1. `parse_instructions` retorna a lista completa (incluindo `('char', '\n')`).
2. `_type_by_chunks` divide a lista em sub-listas por `\n`.
3. Após cada sub-lista, dispara `on_chunk_done` e bloqueia em `_wait_for_enter_or_esc`.
4. ENTER → pressiona Enter no teclado e avança para a próxima linha; ESC → `request_stop()`.

Fluxo de Estados



Compatibilidade de Plataformas

Plataforma	Status	Observação
Windows 10/11	Totalmente suportado	Fonte: Consolas
macOS	Suportado	Requer permissão de Acessibilidade; fonte: Menlo
Linux X11	Suportado	Fonte: Monospace
Linux Wayland	Não suportado	Aviso exibido automaticamente; pynput não funciona

Notas de Compatibilidade de Biblioteca

ttkbootstrap >= 1.20.0 — ScrolledText

`ScrolledText` passou a herdar de `Frame` (não mais de `Text`). O widget interno é acessado via `.text`:

```
self._log_text.text.configure(state="disabled")    # correto
self._log_text.text.insert("end", msg)
self._log_text.configure(state="disabled")         # TclError - Frame não tem state
```

Import path: `from ttkbootstrap.widgets.scrolled import ScrolledText`.